

RAPPORT DE PROJET

API d'analyse de sentiments

Détection de discours haineux à partir de tweets



Awounfouet Ange Junior

Ba Ibra-Alpha

Claver Martin

Li David

Client : Daunale Treupe

Année scolaire 2025 – 2026

Sommaire

Table des matières

Sommaire	2
API d'analyse de sentiments.....	3
Comment utiliser l'API	3
Installation	3
Sans Docker — Prérequis.....	3
Clonage du projet.....	3
Paramétrages du serveur SQL	3
Démarrage.....	3
Test	4
Entraînement du modèle.....	5
Matrices de confusion	6
Métriques par classe.....	7
Comparaison avant / après pondération des classes	8
Analyse des performances	8
Forces	8
Faiblesses et biais identifiés	8
Recommandations issues de l'analyse.....	9
Tâche planifiée (CRON).....	9
Entraîner le modèle	10
Ajouter les derniers tweets à la base de données.....	10
Idées d'améliorations	10

API d'analyse de sentiments

Cette application permet de détecter les discours haineux.

Comment utiliser l'API

Installation

Sans Docker — Prérequis

- Avoir **Python 3.12** installé
- Avoir **MySQL 8.0** installé ou une image Docker MySQL lancée

Clonage du projet

```
# Cloner le dépôt
git clone <repo>
cd advanced_algo_final_project

# Installer les dépendances
pip install -r requirements.txt
```

Paramétrages du serveur SQL

Il est nécessaire de configurer un fichier **.env** dans le dossier du projet avec les informations de connexion à la base de données :

```
DB_HOST=localhost ou [IP_DE_LA_MACHINE_MYSQL]
DB_USER=[Nom d'utilisateur MySQL]
DB_PASSWORD=[Mot de passe MySQL]
DB_NAME=socialmetrics
DB_PORT=[PORT d'hébergement du serveur MySQL]
```

Démarrage

```
python app.py
```

Test

Il est possible de tester l'API via l'interface Swagger : <http://localhost:5000/apidocs/>

Request URL

http://localhost:5000/sentiment

Server response

Code	Details
200	Response body <pre>{ "I don't know": 0.09575931206274846, "I love this!": 0.9969312730613182, "This is kinda bad": -0.0063478577325826735, "This is terrible.": -0.9945610262674309 }</pre>  Download

Ou en ligne de commandes :

```
curl -X POST "http://localhost:5000/sentiment" -H "accept: application/json" -H "Content-Type: application/json" -d "[\"I love this!\", \"This is terrible.\"]"
```

La réponse renvoyée est au format JSON avec le payload suivant : `{ 'tweet' : 'score' }`

La gestion des erreurs de payload est effectuée sur l'endpoint `/sentiment` de l'API :

```
if data is None:
    return jsonify({"error": "Invalid or missing JSON payload."}), 400

if not isinstance(data, list):
    return jsonify({"error": "Invalid format. Expected a JSON array (list)."}), 400

if len(data) == 0:
    return jsonify({"error": "The list is empty. Please provide at least one string."}), 400

if not all(isinstance(item, str) for item in data):
    return jsonify({"error": "Invalid format. All elements in the list must be strings."}), 400
```

Entraînement du modèle

Pour juger la qualité du modèle, on utilise la validation croisée.

Le modèle utilise une `LogisticRegression` pour effectuer un apprentissage de classification supervisé

```
vectorizer = TfidfVectorizer(max_features=1000)
X = vectorizer.fit_transform(X_text)

model = LogisticRegression(class_weight="balanced", max_iter=1000)
model.fit(X, y)
```

Pour cela, on utilise la fonction `evaluate_model` qui permet de :

- Entraîner le modèle sur l'ensemble des données
- Valider le modèle sur l'ensemble des données
- Afficher les résultats de la validation croisée
- Sauvegarder le modèle

Notre but est d'avoir une précision et un rappel supérieurs à 0,8, mais principalement une meilleure précision (pour éviter de valider des tweets insultants comme étant positifs).

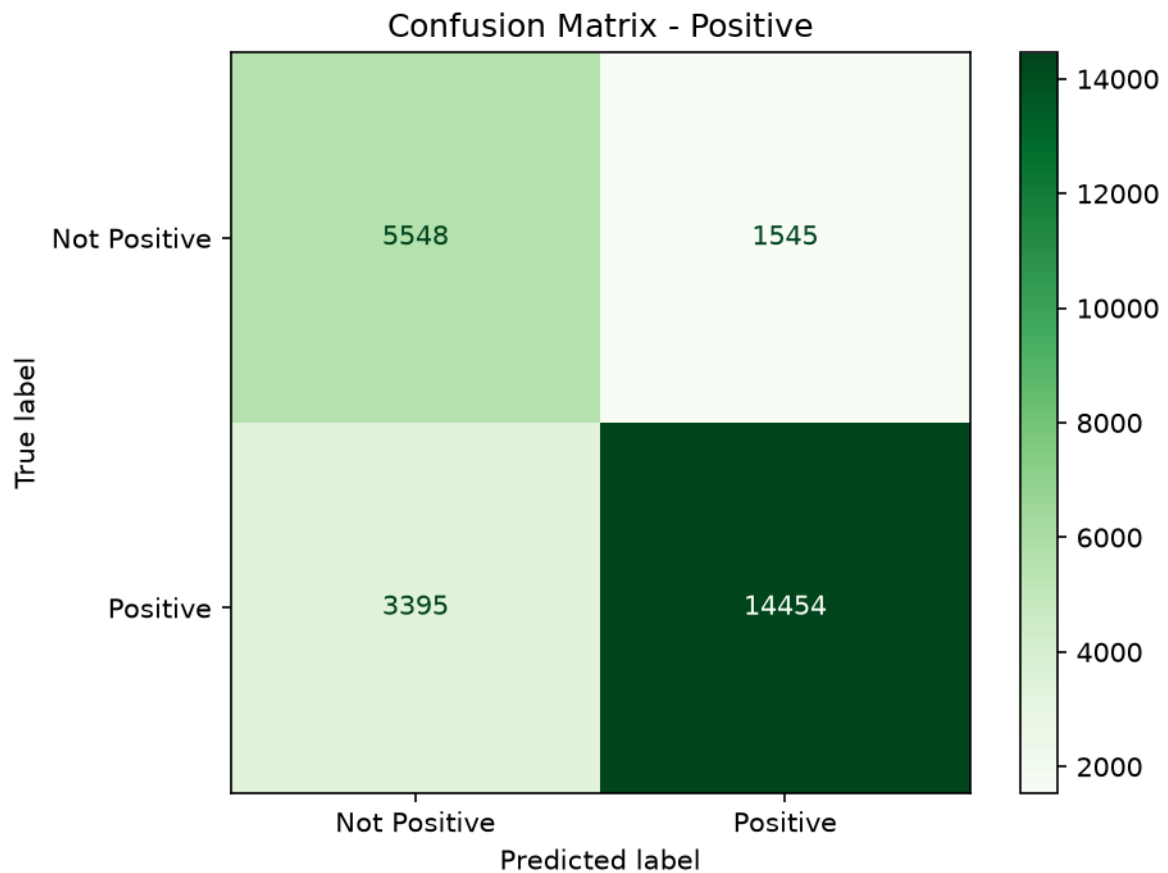
Nous partons d'un jeu de données venant du dataset `tweet_eval_sentiment` sur HuggingFace; 5000 tweets sont chargés en base de données lors de l'initialisation de l'application.

Après l'entraînement sur 11 000 tweets, nous obtenons les métriques suivantes :

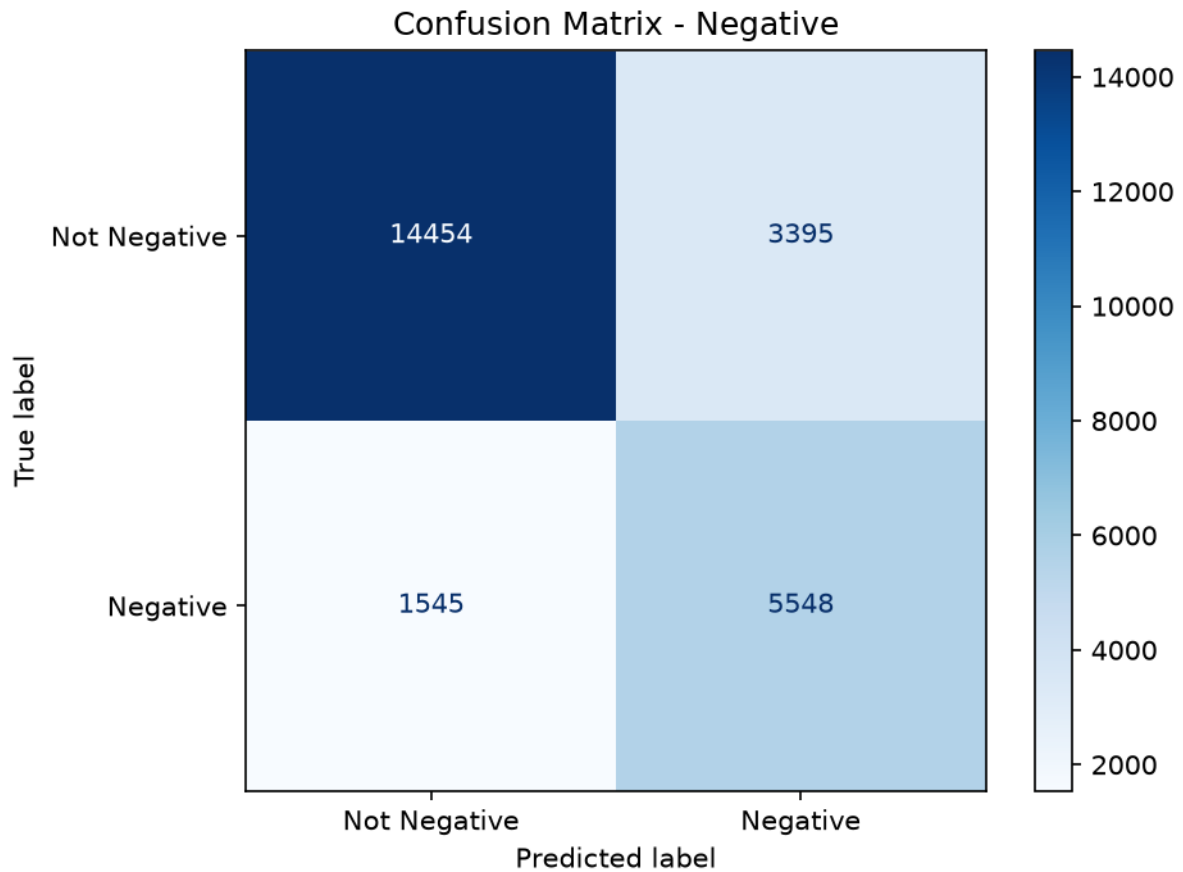
Métrique	Valeur	Écart-type (±)
Accuracy	0,809	0,006
Précision	0,820	0,006
Rappel	0,937	0,007
F1-score	0,875	0,004

Ces résultats reposent toutefois sur un jeu de données relativement simple : il faudrait le complexifier davantage.

Matrices de confusion



Sur l'ensemble complet de la base ($n = 24\,942$ tweets, comparable au jeu « avant » de $24\,934$ tweets), la matrice « Positive » montre $14\,454$ tweets réellement positifs bien détectés sur $17\,849$ (rappel de $81,0\%$), et **1 545** tweets non positifs classés à tort comme positifs (précision de $90,4\%$, en nette hausse par rapport aux $83,9\%$ d'avant pondération).



Sur la matrice « Negative », l'amélioration se confirme à grande échelle : **5 548** des 7 093 tweets réellement négatifs sont détectés (rappel de **78,2 %**, contre 54,8 % avant pondération). La précision négative baisse à 62,0 % (contre 77,0 % avant) : le modèle déclenche plus de fausses alertes sur des tweets qui ne sont pas réellement négatifs, mais dans une proportion plus mesurée qu'observé sur le premier test à 5 000 tweets.

Métriques par classe

Classe	Précision	Rappel	F1-score
Positive	0,904	0,810	0,854
Négative	0,620	0,782	0,692

L'accuracy globale calculée sur l'ensemble complet de la base ($n = 24\,942$ tweets) est de 80,2 %, à comparer aux 82,5 % obtenus avant l'ajout de la pondération de classes ($n = 24\,934$). Le volume de données étant désormais quasiment identique des deux côtés la comparaison est directe et fiable.

Comparaison avant / après pondération des classes

Métrique	Avant (sans pondération, n=24 934)	Après (class_weight='balanced', n=24 942)
Accuracy	0,825	0,802
Précision positive	0,839	0,904
Rappel positif	0,935	0,810
F1 positif	0,885	0,854
Précision négative	0,770	0,620
Rappel négatif	0,548	0,782
F1 négatif	0,640	0,692

En vert : amélioration liée à l'objectif du projet (moins de tweets insultants validés comme positifs). En rouge : dégradation, compromis attendu d'un rééquilibrage de classes.

Analyse des performances

Forces

Confirmé sur l'ensemble de la base (n = 24 942, soit un volume comparable au « avant »), le rappel négatif progresse fortement : de 54,8 % à **78,2 %**. Le modèle manque désormais un peu moins d'un tweet négatif sur cinq, contre un sur deux auparavant. La précision positive s'améliore aussi nettement (83,9 % → 90,4 %) : moins de tweets réellement négatifs se retrouvent validés comme positifs — l'objectif prioritaire du projet. Cette évaluation à grande échelle confirme la tendance déjà observée sur l'échantillon de 5 000 tweets, avec un rappel négatif encore meilleur (78,2 % contre 73,5 %), probablement grâce au volume d'entraînement plus important.

Faiblesses et biais identifiés

Le coût mesuré à grande échelle est plus modéré qu'estimé initialement : l'accuracy globale baisse de 82,5 % à **80,2 %** (soit -2,3 points, contre -4,7 points sur le petit échantillon de 5 000 tweets), et la précision négative recule à 62,0 % (contre 77,0 % avant). Le rappel positif diminue également (93,5 % → 81,0 %).

C'est le compromis habituel d'un rééquilibrage de classes : on échange de la performance sur la classe majoritaire (positive) contre plus d'équité entre les deux classes. Ce compromis est cohérent avec l'objectif du projet, qui privilégie la détection des contenus insultants plutôt que l'accuracy brute — et le coût réel, mesuré sur un volume de données représentatif, est finalement plus faible que ce que laissait supposer le premier test.

Le rappel négatif reste perfectible (78,2 %) : environ un tweet négatif sur cinq échappe encore au modèle. Le déséquilibre de base du jeu de données subsiste (7 093 tweets négatifs pour 17 849 positifs, soit environ 28 % / 72 %, stable par rapport aux évaluations précédentes) — la pondération compense l'effet du déséquilibre sur l'apprentissage, mais n'augmente pas la quantité d'information disponible sur la classe négative.

Recommandations issues de l'analyse

- Conserver `class_weight='balanced'` : son bénéfice est désormais confirmé sur l'ensemble de la base (+23,4 points de rappel négatif, +6,5 points de précision positive, pour un coût de seulement -2,3 points d'accuracy).
- Si le rappel négatif doit encore progresser au-delà de 78,2 %, envisager un ré-échantillonnage complémentaire (sur-échantillonnage des tweets négatifs, ex. SMOTE) en plus de la pondération, en surveillant que la précision négative (62,0 %) ne se dégrade pas davantage.
- Ajuster le seuil de décision (au lieu de 0,5 par défaut) pour regagner du rappel négatif sans nécessairement perdre plus de précision.
- Conserver ce jeu de test à grande échelle ($n \approx 24\,900$) comme référence pour les prochaines itérations, afin de continuer à comparer les configurations de manière rigoureuse d'une semaine à l'autre.
- Poursuivre la collecte de tweets négatifs via le CRON hebdomadaire pour réduire le déséquilibre de base, stable à environ 28 % de négatifs contre 72 % de positifs depuis le début du projet.

Tâche planifiée (CRON)

Afin de mieux peupler la base de données, on ajoute une tâche CRON qui récupère les derniers tweets sur les réseaux sociaux et les ajoute à la base de données.

- Toutes les semaines, le modèle récupère 2 000 nouveaux tweets depuis le modèle `tweet_eval_sentiment` sur HuggingFace.
- Seuls les tweets non présents en doublon sont chargés en base de données.
- Le modèle est ensuite automatiquement ré-entraîné avec les nouvelles données

```
def fetch_and_train_job():
    fetch_and_store_tweets(2000, "tweet_eval_sentiment")
    train_and_save_model()

def run_fetch_scheduler():
    print("Scheduler thread started")

    schedule.every(1).weeks.do(fetch_and_train_job)
    print(schedule.jobs)
    while True:
        print("Checking pending jobs ...")
        schedule.run_pending()
        time.sleep(86400) # Sleep for 1 day
    return go(f, seed, [])
```

Entraîner le modèle

```
python ml/evaluate.py
```

Ajouter les derniers tweets à la base de données

```
python utils/fetch_tweets.py
```

Idées d'améliorations

- Nettoyage du texte avant vectorisation
- Vérification des valeurs vides
- Augmentation du jeu de données : seuls 46 000 tweets sont actuellement disponibles, il faudrait l'étendre pour obtenir un meilleur entraînement du modèle
- Importer des jeux de données de tweets difficiles à classer (mentions, emojis, fautes d'orthographe, langage propre aux tweets) — des sous-jeux existent dans `tweet_eval_sentiment` (voir https://huggingface.co/datasets/cardiffnlp/tweet_eval)
- Traduction / prise en charge de tweets en différentes langues
- Ajout d'un niveau d'intensité : les colonnes en base indiquent aujourd'hui « positif » ou « négatif » ; il faudrait nuancer avec une échelle de -2 à 2 (très négatif, négatif, neutre, positif, très positif)